S20 - EA2. CREACIÓN DE REPOSITORIO

JHOAN SEBASTIAN BASTIDAS DIAZ

INGENIERÍA DE SOFTWARE Y DATOS

JORGE ARMANDO JULIO CRUZ

PROGRAMACIÓN ORIENTADA A OBJETOS II

INSTITUCIÓN UNIVERSITARIA DIGITAL DE ANTIOQUIA

06/11/2026

INTRODUCCION

La Alcaldía de TecnoValle, una ciudad latinoamericana en proceso de transformación digital, moviliza a más de 450.000 pasajeros por día y genera entre 6 y 12 millones de registros diarios provenientes de validadores de tarjetas, sensores de estaciones, aplicaciones móviles y dispositivos IoT instalados en los buses. La plataforma actual procesa esta información de manera imperativa y secuencial, lo que produce demoras de hasta dos horas, errores de conteo por condiciones de carrera y un código extenso y difícil de mantener.

Como respuesta, el proyecto TecnoMovilData busca evolucionar hacia una arquitectura apoyada en la programación funcional. El presente informe documenta el diseño y la implementación en Java.

El documento presenta los objetivos de la actividad, el desarrollo de la solución, el código fuente completo, la evidencia de ejecución, los enlaces de los repositorios y las conclusiones.

OBJETIVOS

- Aplicar conceptos de programación funcional en Java para procesar un gran volumen de datos simulados de transporte urbano, como parte del proyecto TecnoMovil Data.
- Experimentar con funciones puras, expresiones lambda y operaciones de orden superior.
- Implementar las seis operaciones solicitadas: afluencia por estación, horas pico, rutas más utilizadas, patrones de viaje por usuario, tiempo promedio entre estaciones y detección de sobrecarga en rutas.
- Garantizar la inmutabilidad de los datos.

DESARROLLO

La solución se organizó en un único paquete con cuatro clases, cada una con una responsabilidad clara.

CLASE	RESPONSABILIDAD
Registro	Representa un registro del sistema (idUsuario, ruta, estación, acción y timestamp). Es inmutable: sus atributos son final y no tiene métodos set.
GeneradorDatos	Simula los datos del caso con una semilla fija, de modo que siempre se obtienen los mismos registros. Devuelve una lista inmutable.
ProcesadorTransporte	Contiene las operaciones funcionales y la función de orden superior contarPor.
Main	Ejecuta el procesamiento, imprime los resultados y comprueba la inmutabilidad.

Cada registro contiene el identificador del usuario, la ruta, la estación, la acción ("entrada" o "salida") y la marca de tiempo. El generador crea 50.000 usuarios que realizan entre uno y tres viajes cada uno, cada viaje produce un registro de entrada y uno de salida, para un total de 199.646 registros simulados.

Funciones implementadas:

a) **Afluencia por estación:**
   Se filtran los registros de entrada y se agrupan por estación contando cuántos hay en cada grupo.

b) **Identificación de horas pico:**
   Se agrupan todos los registros por la hora del día y luego se ordenan los grupos de mayor a menor flujo y se toman los primeros.

c) **Rutas más utilizadas:**
   Se cuentan las entradas por ruta con la misma función de orden superior y se ordenan de forma descendente para obtener las rutas de mayor volumen.

d) **Patrones de viaje por usuario:**
   Se agrupan los registros por usuario y se conserva únicamente la estación de cada registro, respetando el orden en que aparecen en los datos.

e) **Tiempo promedio entre estaciones:**
   Para cada usuario se ordena una copia de sus registros por fecha y se recorren los pares consecutivos. Cuando el primer registro es una entrada y el segundo una salida, se calcula la diferencia en minutos y se asocia al tramo "estación origen -> estación destino". Finalmente se agrupan los tramos y se obtiene el promedio.

f) **Detección de sobrecarga en rutas:**
   Se toma como ocupación simulada el número de entradas de cada ruta. Si supera el umbral definido, la ruta se marca como "CRITICA", de lo contrario, como "NORMAL".

CODIGO FUENTE EN JAVA

Código 1

Archivo Registro.java

```
package com.tecnomovil;

import java.time.LocalDateTime;


// Esta clase representa un registro del sistema de transporte.
// Todos los atributos son "final": (inmutabilidad).
public class Registro {


   private final String idUsuario;
   private final String ruta;
   private final String estacion;
   private final String accion;          // "entrada" o "salida"
   private final LocalDateTime timestamp;


   public Registro(String idUsuario,
                   String ruta,
                   String estacion,
                   String accion,
                   LocalDateTime timestamp) {
      this.idUsuario = idUsuario;
      this.ruta = ruta;
      this.estacion = estacion;
      this.accion = accion;
      this.timestamp = timestamp;
   }


   // Solo hay getters, por eso el objeto no se puede modificar
   public String getIdUsuario() { return idUsuario; }
   public String getRuta() { return ruta; }
   public String getEstacion() { return estacion; }
   public String getAccion() { return accion; }
   public LocalDateTime getTimestamp() { return timestamp; }
```

```
    // Metodo auxiliar para saber si el registro es una entrada
    public boolean esEntrada() {
        return accion.equals("entrada");
    }
    @Override
    public String toString() {
        return idUsuario + " | " + ruta + " | " + estacion + " | " + accion + " | " + timestamp;
    }
}
```

Código 2

Archivo GeneradorDatos.java

```
package com.tecnomovil;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Random;

// Esta clase solo sirve para SIMULAR datos.
public class GeneradorDatos {

    private static final String[] RUTAS = {"R1", "R2", "R3", "R4", "R5", "A1", "A2"};
    private static final String[] ESTACIONES = {"Central", "Norte", "Sur", "Universidad",
            "Hospital", "Terminal", "Parque", "Centro"};

    public static List<Registro> generar(int cantidadUsuarios) {
        Random random = new Random(42); // semilla fija: siempre salen los mismos datos
        List<Registro> lista = new ArrayList<>();

        for (int i = 0; i < cantidadUsuarios; i++) {
            String idUsuario = "U" + i;
```

```
            int viajes = 1 + random.nextInt(3); // cada usuario hace entre 1 y 3 viajes

            for (int v = 0; v < viajes; v++) {
                String ruta = RUTAS[random.nextInt(RUTAS.length)];
                String origen = ESTACIONES[random.nextInt(ESTACIONES.length)];
                String destino = ESTACIONES[random.nextInt(ESTACIONES.length)];

                // Hora entre las 5 y las 22
                LocalDateTime horaEntrada = LocalDateTime.of(2026, 11, 3,
                        5 + random.nextInt(18), random.nextInt(60));
                // El viaje dura entre 5 y 45 minutos
                LocalDateTime horaSalida = horaEntrada.plusMinutes(5 + random.nextInt(40));

                lista.add(new Registro(idUsuario, ruta, origen, "entrada", horaEntrada));
                lista.add(new Registro(idUsuario, ruta, destino, "salida", horaSalida));
            }
        }
        // Devolvemos la lista INMUTABLE
        return Collections.unmodifiableList(lista);
    }
}
```

Código 3

Archivo ProcesadorTransporte.java

```
package com.tecnomovil;

import java.util.AbstractMap;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.function.Function;
import java.util.function.Predicate;
```

```
import java.util.stream.Collectors;
import java.util.stream.IntStream;

// Aquí están todas las operaciones funcionales.
// Todos los métodos son FUNCIONES PURAS:
public class ProcesadorTransporte {

   // FUNCIÓN DE ORDEN SUPERIOR: recibe otras funciones como parámetros
   public static <K> Map<K, Long> contarPor(List<Registro> registros,
                                    Predicate<Registro> filtro,
                                    Function<Registro, K> agrupador) {
      Map<K, Long> resultado = registros.stream()        // 1. se crea el stream
            .filter(filtro)                          // 2. se dejan solo los registros que cumplen
            .collect(Collectors.groupingBy(agrupador,   // 3. se agrupan por la clave
                  Collectors.counting()));           //    y se cuenta cuántos hay en cada grupo
      return Collections.unmodifiableMap(resultado);      // 4. se devuelve un mapa que no se puede modificar
   }

   // a) Afluencia por estación: cuántos usuarios entran a cada estación
   public static Map<String, Long> afluenciaPorEstacion(List<Registro> registros) {
      return contarPor(registros, Registro::esEntrada, Registro::getEstacion);
   }

   // b) Horas pico: se agrupa por hora del día y se toman las de mayor flujo
   public static List<Map.Entry<Integer, Long>> horasPico(List<Registro> registros, int cuantas) {
      // Lambda: r -> true (acepta todos los registros)  y  r -> r.getTimestamp().getHour() (saca la hora)
      Map<Integer, Long> flujoPorHora = contarPor(registros,
            r -> true,
            r -> r.getTimestamp().getHour());

      return flujoPorHora.entrySet().stream()
            .sorted((a, b) -> Long.compare(b.getValue(), a.getValue())) // de mayor a menor
            .limit(cuantas)                                  // solo las primeras
            .collect(Collectors.toList());
   }
```

```
// c) Rutas más utilizadas: se cuentan las entradas por ruta y se ordenan de mayor a menor
public static List<Map.Entry<String, Long>> rutasMasUtilizadas(List<Registro> registros, int cuantas) {
    return contarPor(registros, Registro::esEntrada, Registro::getRuta)
            .entrySet().stream()
            .sorted((a, b) -> Long.compare(b.getValue(), a.getValue()))
            .limit(cuantas)
            .collect(Collectors.toList());
}

// d) Patrones de viaje: por cada usuario, la lista de estaciones en el orden de los registros
public static Map<String, List<String>> patronesDeViaje(List<Registro> registros) {
    Map<String, List<String>> patrones = registros.stream()
            .collect(Collectors.groupingBy(Registro::getIdUsuario,      // agrupa por usuario
                    Collectors.mapping(Registro::getEstacion,           // de cada registro toma solo la estación
                            Collectors.toList())));                     // las guarda en una lista
    return Collections.unmodifiableMap(patrones);
}

// e) Tiempo promedio (en minutos) entre estaciones consecutivas
// Se compara cada registro de entrada con el registro siguiente del mismo usuario.
public static Map<String, Double> tiempoPromedioEntreEstaciones(List<Registro> registros) {
    // Primero se agrupan los registros por usuario
    Map<String, List<Registro>> porUsuario = registros.stream()
            .collect(Collectors.groupingBy(Registro::getIdUsuario));

    Map<String, Double> promedios = porUsuario.values().stream()
            .flatMap(registrosDelUsuario -> {
                // ordena una copia por hora, la lista original no cambia
                List<Registro> ordenados = registrosDelUsuario.stream()
                        .sorted(Comparator.comparing(Registro::getTimestamp))
                        .collect(Collectors.toList());

                // Se recorren los pares consecutivos
                return IntStream.range(1, ordenados.size())
```

```
                .filter(i -> ordenados.get(i - 1).esEntrada()          // el anterior es entrada
                        && !ordenados.get(i).esEntrada())              // y el actual es salida
                .mapToObj(i -> {
                    Registro entrada = ordenados.get(i - 1);
                    Registro salida = ordenados.get(i);
                    String tramo = entrada.getEstacion() + " -> " + salida.getEstacion();
                    double minutos = java.time.Duration
                            .between(entrada.getTimestamp(), salida.getTimestamp())
                            .toMinutes();
                    return new AbstractMap.SimpleEntry<>(tramo, minutos);
                });
        })
        // Se agrupan por tramo y se saca el promedio de los minutos
        .collect(Collectors.groupingBy(Map.Entry::getKey, TreeMap::new,
                Collectors.averagingDouble(Map.Entry::getValue)));

    return Collections.unmodifiableMap(promedios);
  }

  // f) Sobrecarga: si las entradas de una ruta superan el umbral, la ruta es "CRITICA"
  public static Map<String, String> estadoDeRutas(List<Registro> registros, long umbral) {
    Map<String, String> estados = contarPor(registros, Registro::esEntrada, Registro::getRuta)
            .entrySet().stream()
            .collect(Collectors.toMap(
                    Map.Entry::getKey,                                 // la clave es el nombre de la ruta
                    e -> e.getValue() > umbral ? "CRITICA" : "NORMAL",          // lambda que decide el estado
                    (a, b) -> a,
                    TreeMap::new));
    return Collections.unmodifiableMap(estados);
  }
}
```

Código 4

Archivo Main.java

```
package com.tecnomovil;


import java.util.List;

import java.util.Map;


public class Main {

   public static void main(String[] args) {

      // Se generan los datos simulados

      List<Registro> registros = GeneradorDatos.generar(50000);

      int tamanoInicial = registros.size();

      System.out.println("=== TecnoMovil Data ===");

      System.out.println("Total de registros: " + tamanoInicial);


      // a) Afluencia por estación

      System.out.println("\n--- a) Afluencia por estacion ---");

      Map<String, Long> afluencia = ProcesadorTransporte.afluenciaPorEstacion(registros);

      // forEach con lambda: (clave, valor) -> imprimir

      afluencia.forEach((estacion, total) -> System.out.println(estacion + ": " + total));


      // b) Horas pico

      System.out.println("\n--- b) Top 3 horas pico ---");

      ProcesadorTransporte.horasPico(registros, 3)

            .forEach(e -> System.out.println(e.getKey() + ":00 -> " + e.getValue() + " registros"));


      // c) Rutas más utilizadas

      System.out.println("\n--- c) Rutas mas utilizadas ---");

      ProcesadorTransporte.rutasMasUtilizadas(registros, 3)

            .forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));


      // d) Patrones de viaje (solo se muestran 5 usuarios)

      System.out.println("\n--- d) Patrones de viaje (5 usuarios) ---");

      Map<String, List<String>> patrones = ProcesadorTransporte.patronesDeViaje(registros);
```

```
        patrones.entrySet().stream()
                .limit(5)
                .forEach(e -> System.out.println(e.getKey() + " -> " + e.getValue()));

        // e) Tiempo promedio entre estaciones (solo 5 tramos)
        System.out.println("\n--- e) Tiempo promedio entre estaciones (5 tramos) ---");
        ProcesadorTransporte.tiempoPromedioEntreEstaciones(registros).entrySet().stream()
                .limit(5)
                .forEach(e -> System.out.printf("%s: %.2f min%n", e.getKey(), e.getValue()));

        // f) Sobrecarga en rutas
        long umbral = 14300;
        System.out.println("\n--- f) Estado de rutas (umbral = " + umbral + ") ---");
        ProcesadorTransporte.estadoDeRutas(registros, umbral)
                .forEach((ruta, estado) -> System.out.println(ruta + ": " + estado));

        // Comprobación de inmutabilidad
        System.out.println("\n--- Inmutabilidad ---");
        System.out.println("La lista original sigue con " + registros.size()
                + " registros");
    }
}
```

CONCEPTOS DE PROGRAMACIÓN FUNCIONAL APLICADOS

Funciones puras:

Todos los métodos de ProcesadorTransporte son funciones puras: su resultado depende únicamente de los parámetros recibidos, no leen ni modifican variables externas, no imprimen información y no alteran la lista original. Por ello, con la misma lista de entrada siempre devuelven el mismo resultado, lo que facilita las pruebas y el mantenimiento.

Inmutabilidad:

La clase Registro declara todos sus atributos como final y no ofrece métodos de modificación. La lista de datos se entrega con Collections.unmodifiableList y cada resultado se devuelve con Collections.unmodifiableMap o como una lista nueva. Además, la final se observa que su tamaño no cambia después del procesamiento.

Funciones de orden superior:

El método genérico contarPor recibe como parámetros un Predicate<Registro> (para filtrar) y una Function<Registro, K> (para agrupar). Gracias a esto, las tareas reutilizan la misma lógica de conteo cambiando únicamente las funciones que se le pasan.

Expresiones lambda:

Se utilizaron lambdas como r -> true, r -> r.getTimestamp().getHour(), (a, b) -> Long.compare(b.getValue(), a.getValue()) y e -> e.getValue() > umbral ¿ “CRITICA” : “NORMAL”. También se emplearon referencias a métodos como Registro::esEntrada, Registro::getEstacion y Registro::getRuta, que son una forma abreviada de escribir lambdas simples.

Streams:

El procesamiento se construyó con pipelines declarativos: stream() para crear el flujo, operaciones intermedias y operaciones terminales con collect.

EVIDENCIA DE EJECUCION

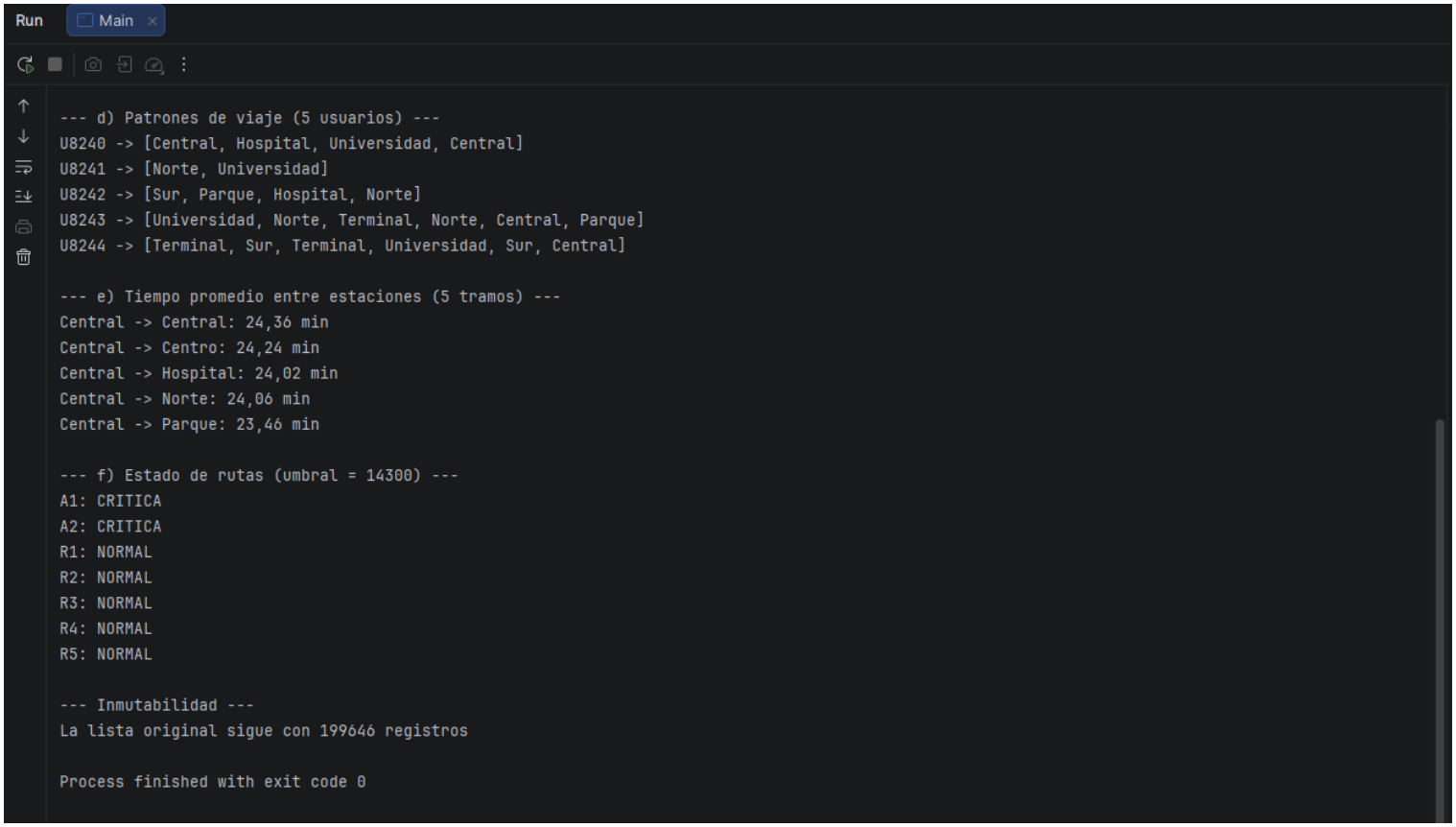


```
Run  Main
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.4\lib\idea_rt.jar=51515" -Dfile.encoding
=== TecnoMovil Data ===
Total de registros: 199646

--- a) Afluencia por estacion ---
Parque: 12469
Sur: 12433
Centro: 12505
Universidad: 12412
Central: 12549
Hospital: 12490
Norte: 12633
Terminal: 12332

--- b) Top 3 horas pico ---
13:00 -> 11300 registros
8:00 -> 11276 registros
17:00 -> 11216 registros

--- c) Rutas mas utilizadas ---
A1: 14410
A2: 14389
R1: 14277
```



```
Run  Main

--- d) Patrones de viaje (5 usuarios) ---
U8240 -> [Central, Hospital, Universidad, Central]
U8241 -> [Norte, Universidad]
U8242 -> [Sur, Parque, Hospital, Norte]
U8243 -> [Universidad, Norte, Terminal, Norte, Central, Parque]
U8244 -> [Terminal, Sur, Terminal, Universidad, Sur, Central]

--- e) Tiempo promedio entre estaciones (5 tramos) ---
Central -> Central: 24,36 min
Central -> Centro: 24,24 min
Central -> Hospital: 24,02 min
Central -> Norte: 24,06 min
Central -> Parque: 23,46 min

--- f) Estado de rutas (umbral = 14300) ---
A1: CRITICA
A2: CRITICA
R1: NORMAL
R2: NORMAL
R3: NORMAL
R4: NORMAL
R5: NORMAL

--- Inmutabilidad ---
La lista original sigue con 199646 registros

Process finished with exit code 0
```

CONCLUSIONES

- Se logró aplicar la programación funcional en un escenario realista: el módulo procesa cerca de 200.000 registros simulados y resuelve las seis tareas solicitadas.

- El uso de funciones puras, lambdas y funciones de orden superior favoreció la reutilización: una sola función sirvió de base para varias tareas, cambiando únicamente los criterios de filtrado y agrupación.

- Finalmente, la solución cumple con los objetivos de la actividad y queda alojada en repositorios de GitHub

ENLACES

GitHub: https://github.com/bastidasd11/TecnoMovilData_BastidasJhoan.git